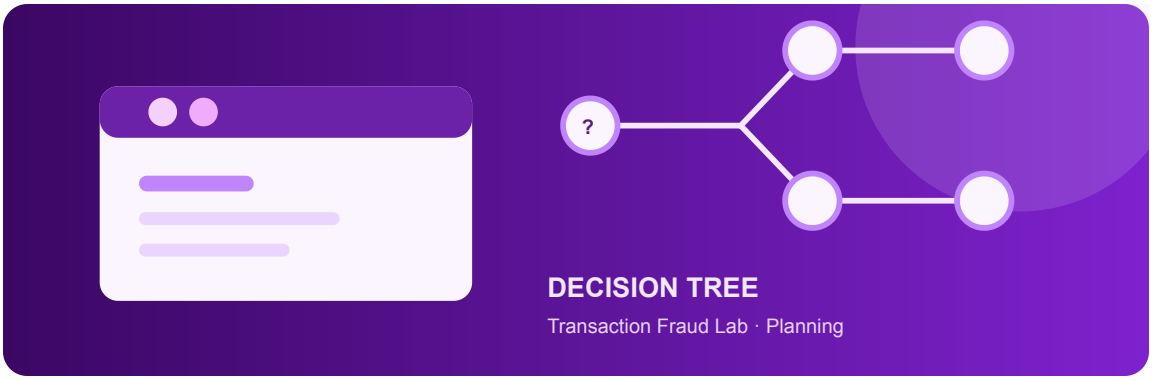
💸 Transaction Fraud Classification



Decision Tree · A book-ready, reproducible walkthrough

This chapter builds an educational fraud-classification pipeline from first principles. We will define the problem, create a deterministic real dataset, explain the mathematics of decision trees, prevent leakage, train and evaluate a model, choose an operating threshold, save a deployable artifact, and finish with a practical checklist.

> **Safety and scope:** the labels in the default dataset are synthetic. They demonstrate workflow and model behaviour; they are not evidence that the model can detect real fraud. Never automate a financial decision from this notebook. Real use requires legal review, privacy safeguards, human review, monitoring, fairness evaluation and incident response.

🎯 Learning objectives

By the end of this chapter you should be able to:

1. Explain classification, class imbalance and why accuracy can mislead.
2. Derive the Gini impurity and information-gain ideas used by a Decision Tree.
3. Generate and inspect a reproducible transaction dataset without target leakage.
4. Build a preprocessing-and-model pipeline with a stratified holdout set.
5. Read a confusion matrix and calculate precision, recall, F1 and PR-AUC.
6. Select a threshold using an explicit business-cost assumption.
7. Save, reload and test the complete pipeline for a Streamlit application.

🧭 Real-world use cases (and boundaries)

Fraud models are commonly used as **decision-support signals**, not as final verdicts:

- **Card authorisation:** route a high-risk payment to step-up authentication.
- **E-commerce review queues:** prioritise orders for manual review before fulfilment.
- **Account protection:** trigger additional verification after an unusual login or transfer.
- **Claims triage:** rank cases for investigation while preserving a human appeal path.

A model should not silently block a person, close an account, or deny a claim. The model only estimates a pattern learned from historical labels, which may contain reporting bias and investigator bias.

📚 Real dataset and provenance

This chapter uses the real **Credit Card Fraud Detection** dataset published by the ULB Machine Learning Group through Kaggle. It contains 284,807 historical transactions, anonymised PCA features `V1` – `V28`, `Time`, `Amount`, and the binary `Class` target. The fraud rate is below 0.2%, so accuracy alone is not meaningful.

The raw CSV is intentionally not committed because it is large and must be downloaded under the publisher's current terms. See `data/README.md` for the reproducible download command. The model uses a time-ordered holdout: the first 80% of rows are training data and the final 20% are an untouched temporal holdout.

This dataset is historical and anonymised. It is suitable for education and benchmarking, not automatic payment decisions or claims about current fraud operations.

🧠 What is a Decision Tree?

A Decision Tree recursively asks questions such as `amount > threshold?` or `channel = online?`. Each internal node is a split, each branch is an answer, and each leaf stores a prediction or class probability.

For a node containing class proportions $p_1, \ldots, p_K$, the **Gini impurity** is

$$G(t) = 1 - \sum_{k=1}^{K} p_k^2.$$

For binary classes, $G(t) = 2p(1-p)$: it is zero when a node is pure and largest at $p = 0.5$. If a parent node t is split into children t_L and t_R, the weighted impurity is

$$G_{split} = \frac{N_L}{N_t} G(t_L) + \frac{N_R}{N_t} G(t_R),$$

so the impurity reduction is

$$\Delta G = G(t) - G_{split}.$$

The tree searches candidate splits and prefers a large impurity reduction. `max_depth`, `min_samples_leaf` and pruning controls limit complexity and reduce memorisation.

🧮 A small split calculation

Suppose a parent has 10 transactions: 4 fraud and 6 legitimate. Then

$$G(parent) = 1 - (0.4^2 + 0.6^2) = 0.48.$$

A proposed split creates a left child with 3 fraud and 1 legitimate and a right child with 1 fraud and 5 legitimate:

$$G(left) = 1 - (0.75^2 + 0.25^2) = 0.375,$$

$$G(right) = 1 - (\frac{1}{6}^2 + \frac{5}{6}^2) \approx 0.278.$$

The weighted child impurity is $\frac{4}{10}(0.375) + \frac{6}{10}(0.278) \approx 0.317$, giving $\Delta G \approx 0.163$. The split makes the children more homogeneous than the parent.

⚖️ Metrics for rare-event classification

Let TP, FP, TN and FN denote true positives, false positives, true negatives and false negatives.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad ext(oftenmisleadingunderimbalance)$$

$$Precision = \frac{TP}{TP + FP}, \qquad Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}.$$

Precision asks “when we flag fraud, how often are we right?” Recall asks “how much fraud did we catch?” The precision-recall curve evaluates these trade-offs across thresholds. **PR-AUC** is usually more informative than ROC-AUC when the positive class is rare.

A probability threshold is an operating policy, not a universal truth. If missing fraud costs C_{FN} and reviewing a legitimate transaction costs C_{FP}, a simple evaluation cost is $C_{FN}FN + C_{FP}FP$. Real teams should use validated, documented costs and monitor them over time.

🛠️ Imports and reproducibility

The core tools are:

- **Python 3.12** — execution environment.
- **pandas / NumPy** — tabular data and numerical simulation.
- **Matplotlib / Seaborn** — publication-friendly charts.
- **scikit-learn** — preprocessing, Decision Tree, metrics and model selection.
- **joblib** — trusted local artifact persistence.
- **Jupyter / nbconvert** — executable notebook and HTML/PDF publishing.
- **Streamlit** — optional educational UI layer after review.

The dependency versions are pinned in this project's `requirements.txt` so the saved artifact and serving environment agree.

In [1]:
```
from pathlib import Path
import json
import hashlib
import platform

import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import sklearn
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    ConfusionMatrixDisplay, accuracy_score, average_precision_score,
    classification_report, confusion_matrix, f1_score, precision_score,
    precision_recall_curve, recall_score, roc_auc_score,
)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree

RANDOM_STATE = 42
ROOT = Path.cwd()
ARTIFACT_DIR = ROOT / "artifacts"
ARTIFACT_DIR.mkdir(exist_ok=True)

print(f"Python: {platform.python_version()}")
print(f"scikit-learn: {sklearn.__version__}")
print(f"Working directory: {ROOT}")
```

```
Python: 3.12.13
scikit-learn: 1.6.1
Working directory: /Users/abhishekpanda/Documents/machine-learning-algorithm
s/Supervised-ML/Classification/Decision-Tree/02-transaction-fraud-classifica
tion
```

1. Load and audit the real fraud dataset

The source columns are `Time`, `V1` – `V28`, `Amount`, and `Class`. The PCA columns are anonymised by the dataset publisher. We do not invent labels or features. A time-ordered split is used because random mixing of future transactions into training can produce optimistic estimates for a live fraud-monitoring workflow.

In [2]:
```
DATA_PATH = ROOT / "data" / "creditcard.csv"
if not DATA_PATH.is_file():
    raise FileNotFoundError(f"Download the dataset first; see {DATA_PATH.par
df = pd.read_csv(DATA_PATH)
expected_columns = ["Time", *[f"V{i}" for i in range(1, 29)], "Amount", "Cla
missing = sorted(set(expected_columns) - set(df.columns))
if missing:
    raise ValueError(f"Dataset is missing columns: {missing}")
df = df[expected_columns].dropna()
df["Class"] = df["Class"].astype("int8")
df.head()
```

Out[2]:

	Time	V1	V2	V3	V4	V5	V6	V7
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941

5 rows × 31 columns

In [3]:
```
print(f"Rows: {len(df):,}")
print(f"Columns: {df.shape[1]}")
print(f"Fraud rate: {df['Class'].mean():.3%}")
df.isna().sum().to_frame("missing_values").T
```

```
Rows: 284,807
Columns: 31
Fraud rate: 0.173%
```

Out[3]:

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V2
missing_values	0	0	0	0	0	0	0	0	0	0	...	0	0	0	

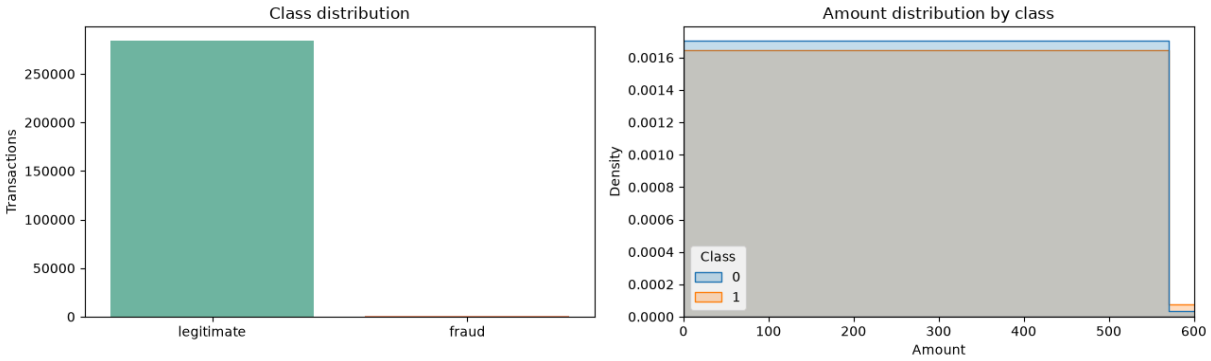
1 rows × 31 columns

First inspection

The fraud class is extremely rare. Always compare the positive-class rate with model metrics; a model that predicts “legitimate” for every row can have high accuracy while

catching zero fraud.

In [4]:
```
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
sns.countplot(data=df, x="Class", ax=axes[0], hue="Class", legend=False, pal
axes[0].set_title("Class distribution")
axes[0].set_xticks([0, 1], ["legitimate", "fraud"])
axes[0].set_xlabel("")
axes[0].set_ylabel("Transactions")
sns.histplot(data=df, x="Amount", hue="Class", bins=45, element="step", stat
axes[1].set_title("Amount distribution by class")
axes[1].set_xlim(0, 600)
plt.tight_layout()
```



2. Define features and protect against leakage

The target is `Class`. All predictors are fields supplied by the source dataset. The time-ordered split keeps the final period untouched for evaluation. In a production project, the feature-availability timestamp and any post-authorisation fields would require a separate data contract review.

In [5]:
```
TARGET = "Class"
FEATURES = ["Time", *[f"V{i}" for i in range(1, 29)], "Amount"]
X = df[FEATURES].copy()
y = df[TARGET].copy()

split = int(len(df) * 0.80)
X_train, X_test = X.iloc[:split].copy(), X.iloc[split:].copy()
y_train, y_test = y.iloc[:split].copy(), y.iloc[split:].copy()
numeric_features = FEATURES
print(f"Train rows: {len(X_train):,}; holdout rows: {len(X_test):,}")
print(f"Train fraud rate: {y_train.mean():.3%}")
print(f"Holdout fraud rate:  {y_test.mean():.3%}")
```

```
Train rows: 227,845; holdout rows: 56,962
Train fraud rate: 0.183%
Holdout fraud rate:  0.132%
```

3. Build a leakage-safe numeric pipeline

The ULB dataset already supplies numeric anonymised PCA components. Median imputation remains inside the pipeline so the serving contract is explicit, and the estimator is fitted only on the historical training period. Saving the complete pipeline prevents training and serving transformations from drifting apart.

In [6]:
```
numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
])
preprocessor = ColumnTransformer(transformers=[
    ("numeric", numeric_pipeline, numeric_features),
])

baseline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", DummyClassifier(strategy="most_frequent")),
])
baseline.fit(X_train, y_train)
baseline_pred = baseline.predict(X_test)
print(classification_report(y_test, baseline_pred, zero_division=0))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56887
           1       0.00      0.00      0.00        75

    accuracy                           1.00     56962
   macro avg       0.50      0.50      0.50     56962
weighted avg       1.00      1.00      1.00     56962
```

4. Train the Decision Tree

`class_weight="balanced"` gives more influence to the minority class during split selection. `max_depth=6` and `min_samples_leaf=30` are deliberately conservative: a deep tree can memorise noise and produce unstable rules. These are teaching defaults, not universal production settings.

In [7]:
```
tree_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", DecisionTreeClassifier(
        criterion="gini", max_depth=8, min_samples_leaf=20,
        class_weight="balanced", random_state=RANDOM_STATE,
    )),
])
tree_pipeline.fit(X_train, y_train)
probability_fraud = tree_pipeline.predict_proba(X_test)[:, list(tree_pipelir
prediction_default = (probability_fraud >= 0.50).astype("int8")
print(classification_report(y_test, prediction_default, zero_division=0))
```

```
              precision    recall  f1-score   support

           0       1.00      0.99      0.99     56887
           1       0.07      0.77      0.13        75

    accuracy                           0.99     56962
   macro avg       0.53      0.88      0.56     56962
weighted avg       1.00      0.99      0.99     56962
```

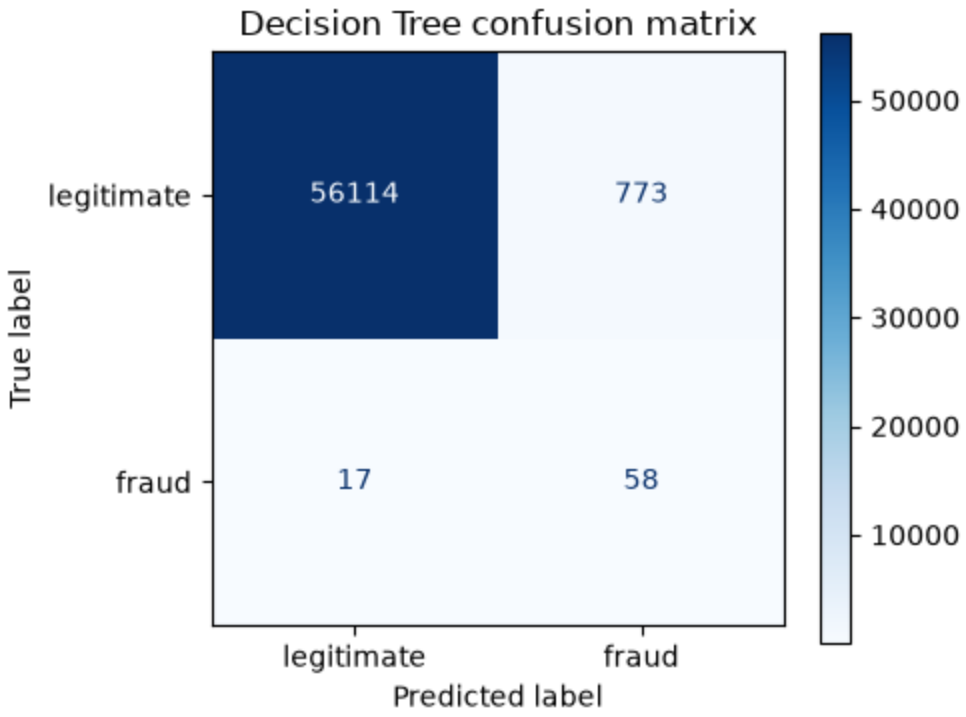
In [8]:
```python
def metric_summary(y_true, predictions, probabilities=None) -> dict[str, flo
    result = {
        "accuracy": accuracy_score(y_true, predictions),
        "precision": precision_score(y_true, predictions, pos_label=1, zero_
        "recall": recall_score(y_true, predictions, pos_label=1, zero_divisi
        "f1": f1_score(y_true, predictions, pos_label=1, zero_division=0),
    }
    if probabilities is not None:
        result["pr_auc"] = average_precision_score(y_true == 1, probabilitie
        result["roc_auc"] = roc_auc_score(y_true == 1, probabilities)
    return result

metrics = pd.DataFrame({
    "baseline": metric_summary(y_test, baseline_pred),
    "decision_tree": metric_summary(y_test, prediction_default, probability_
}).T
metrics.round(3)
```

Out[8]:

	accuracy	precision	recall	f1	pr_auc	roc_auc
baseline	0.999	0.00	0.000	0.000	NaN	NaN
decision_tree	0.986	0.07	0.773	0.128	0.665	0.891

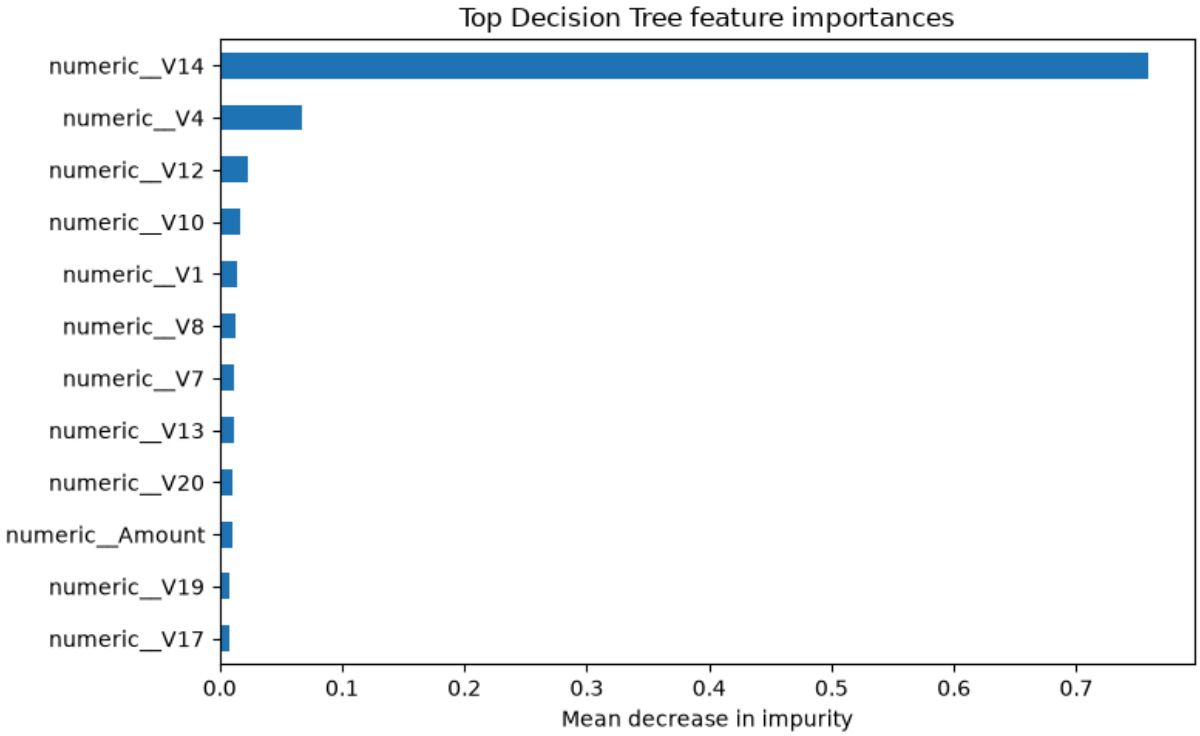
In [9]:
```python
fig, ax = plt.subplots(figsize=(5, 4))
ConfusionMatrixDisplay.from_predictions(
    y_test, prediction_default, labels=[0, 1], display_labels=["legitimate",
)
ax.set_title("Decision Tree confusion matrix")
plt.tight_layout()
```



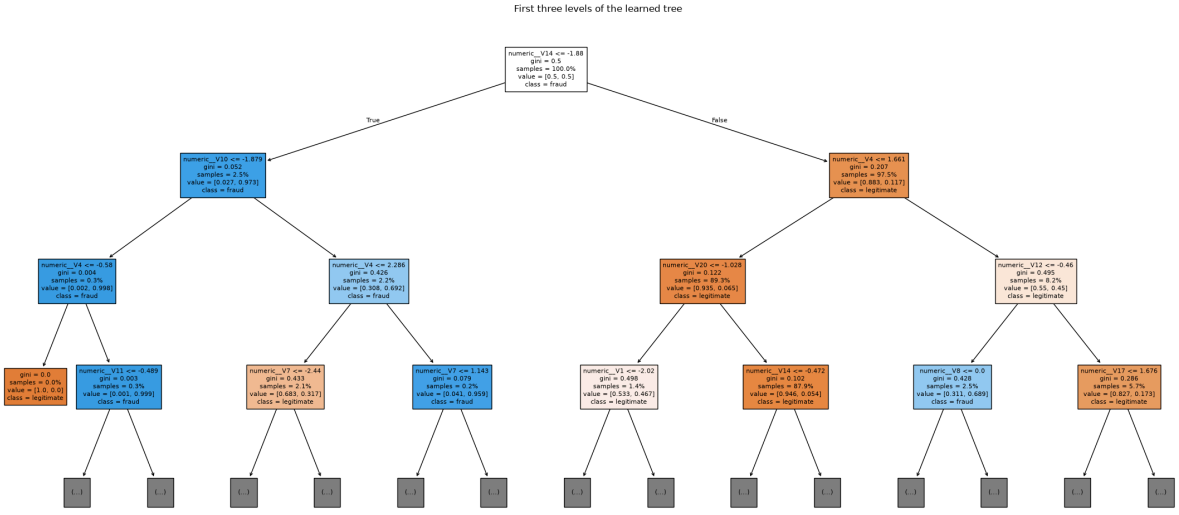
5. Inspect the learned tree

The tree is interpretable as a sequence of conditions, but interpretability does not mean causality. A feature used near the root is useful for separating this sample; it is not proof that the feature causes fraud.

In [10]:
```
classifier = tree_pipeline.named_steps["classifier"]
feature_names = tree_pipeline.named_steps["preprocessor"].get_feature_names_
importance = pd.Series(classifier.feature_importances_, index=feature_names)
importance.head(12).sort_values().plot.barh(figsize=(8, 5), title="Top Decis
plt.xlabel("Mean decrease in impurity")
plt.tight_layout()
```



In [11]:
```
plt.figure(figsize=(22, 10))
plot_tree(
    classifier, feature_names=feature_names, class_names=["legitimate", "fra
    filled=True, max_depth=3, fontsize=8, impurity=True, proportion=True
)
plt.title("First three levels of the learned tree")
plt.tight_layout()
```



6. Choose a threshold with an explicit cost assumption

`predict()` uses a default threshold of 0.50. Fraud operations often choose another threshold because false negatives and false positives have different consequences. Here

we show a transparent illustrative policy:

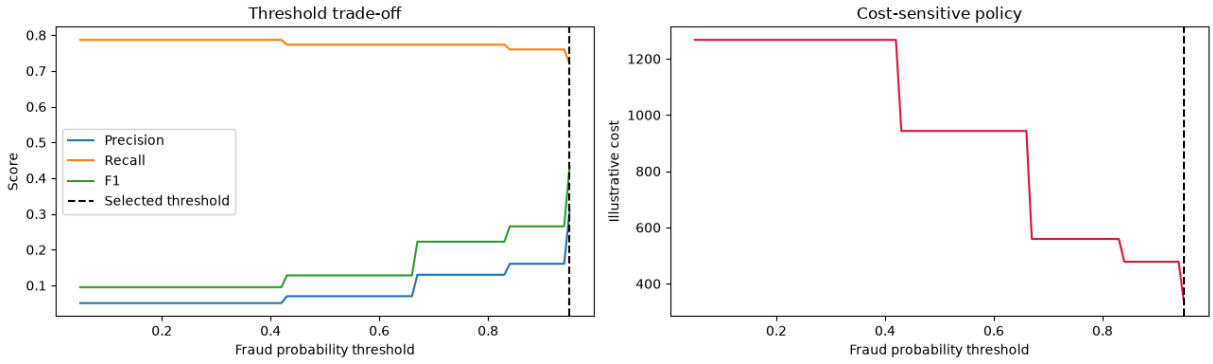
- $C_{FN} = 10$: missing fraud is ten cost units.
- $C_{FP} = 1$: sending a legitimate payment to review is one cost unit.

We select the threshold with the lowest holdout cost only as an educational example. In real work, choose the threshold on a validation set and report performance on a separate untouched test set.

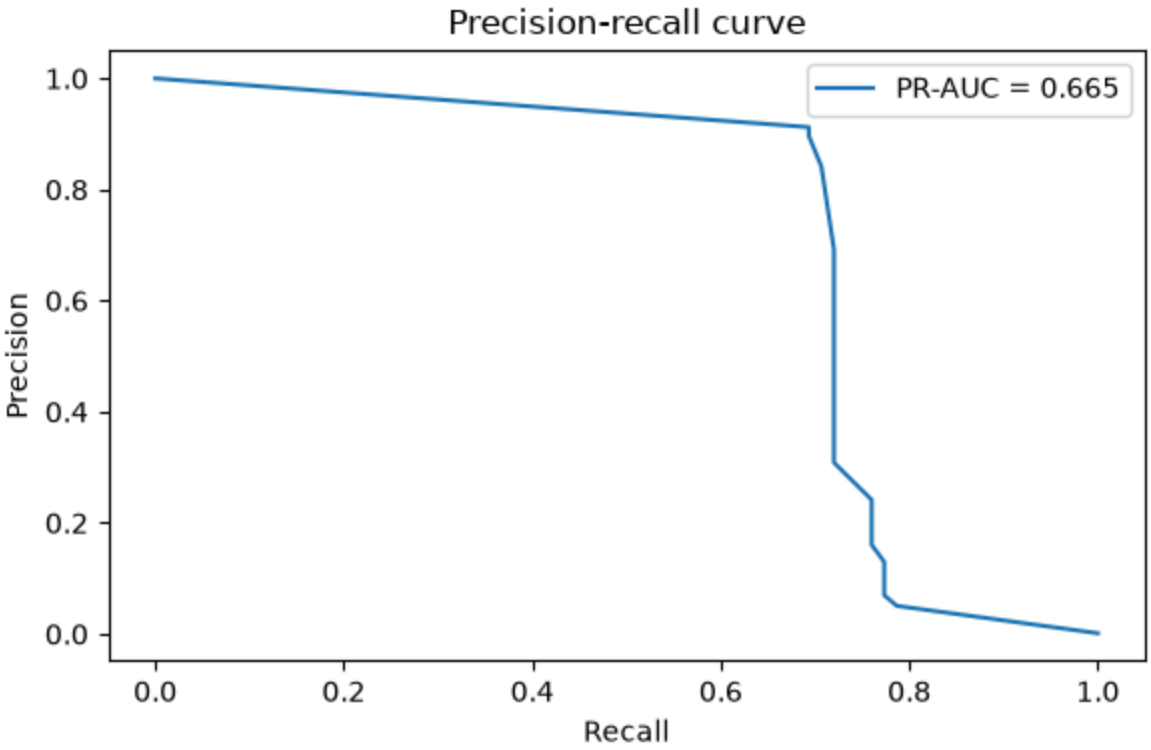
In [12]:
```
thresholds = np.linspace(0.05, 0.95, 91)
rows = []
for threshold in thresholds:
    pred = (probability_fraud >= threshold).astype("int8")
    tn, fp, fn, tp = confusion_matrix(y_test, pred, labels=[0, 1]).ravel()
    rows.append({
        "threshold": threshold, "precision": precision_score(y_test, pred, p
        "recall": recall_score(y_test, pred, pos_label=1, zero_division=0),
        "f1": f1_score(y_test, pred, pos_label=1, zero_division=0),
        "cost": 10 * fn + fp, "fp": fp, "fn": fn,
    })
threshold_table = pd.DataFrame(rows)
best = threshold_table.loc[threshold_table["cost"].idxmin()]
print(best.to_string())

fig, ax = plt.subplots(1, 2, figsize=(13, 4))
ax[0].plot(threshold_table["threshold"], threshold_table["precision"], label
ax[0].plot(threshold_table["threshold"], threshold_table["recall"], label="F
ax[0].plot(threshold_table["threshold"], threshold_table["f1"], label="F1")
ax[0].axvline(best["threshold"], color="black", linestyle="--", label="Selec
ax[0].set(xlabel="Fraud probability threshold", ylabel="Score", title="Thres
ax[0].legend()
ax[1].plot(threshold_table["threshold"], threshold_table["cost"], color="cri
ax[1].axvline(best["threshold"], color="black", linestyle="--")
ax[1].set(xlabel="Fraud probability threshold", ylabel="Illustrative cost",
plt.tight_layout()
```

```
threshold      0.950000
precision      0.308571
recall         0.720000
f1             0.432000
cost         331.000000
fp           121.000000
fn            21.000000
```



In [13]:
```
precision_curve, recall_curve, curve_thresholds = precision_recall_curve(y_t
plt.figure(figsize=(6, 4))
plt.plot(recall_curve, precision_curve, label=f"PR-AUC = {average_precision_
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-recall curve")
plt.legend()
plt.tight_layout()
```



7. Save and reload the complete artifact

Only load artifacts you created or explicitly trust: joblib uses pickle-based serialization and untrusted files can execute code. The metadata records the training contract so a deployment can check its expected target, features and library version.

In [14]:
```
MODEL_PATH = ARTIFACT_DIR / "transaction_fraud_tree_pipeline.joblib"
METADATA_PATH = ARTIFACT_DIR / "model_metadata.json"
joblib.dump(tree_pipeline, MODEL_PATH)
metadata = {
    "algorithm": "DecisionTreeClassifier",
    "project": "transaction-fraud-classification",
    "target_column": TARGET,
    "positive_class": 1,
    "input_features": FEATURES,
    "classes": tree_pipeline.classes_.tolist(),
    "selected_threshold": float(best["threshold"]),
    "false_negative_cost": 10,
    "false_positive_cost": 1,
    "split_strategy": "first 80% by Time for training, final 20% by Time for
```

```
    "train_rows": len(X_train), "test_rows": len(X_test),
    "fraud_rate": float(y.mean()),
    "holdout_pr_auc": float(average_precision_score(y_test, probability_frau
    "holdout_roc_auc": float(roc_auc_score(y_test, probability_fraud)),
    "dataset": {
        "name": "Credit Card Fraud Detection",
        "publisher": "ULB Machine Learning Group",
        "source": "https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud",
        "local_filename": "creditcard.csv",
        "sha256": hashlib.sha256(DATA_PATH.read_bytes()).hexdigest(),
        "redistribution": "Do not redistribute; download under the source te
    },
    "sklearn_version": sklearn.__version__,
    "python_version": platform.python_version(),
    "educational_only": True,
}
METADATA_PATH.write_text(json.dumps(metadata, indent=2) + "\n", encoding="ut
loaded = joblib.load(MODEL_PATH)
assert loaded.predict(X_test.head(5)).shape == (5,)
assert json.loads(METADATA_PATH.read_text())["input_features"] == FEATURES
print(f"Saved {MODEL_PATH}")
print(f"Saved {METADATA_PATH}")
print("Round-trip artifact test passed.")
```

```
Saved /Users/abhishekpanda/Documents/machine-learning-algorithms/Supervised-
ML/Classification/Decision-Tree/02-transaction-fraud-classification/artifact
s/transaction_fraud_tree_pipeline.joblib
Saved /Users/abhishekpanda/Documents/machine-learning-algorithms/Supervised-
ML/Classification/Decision-Tree/02-transaction-fraud-classification/artifact
s/model_metadata.json
Round-trip artifact test passed.
```

8. Example serving prediction

A Streamlit app should validate inputs, load the artifact once with `st.cache_resource`, display the probability and threshold, and clearly state that the result is a review signal—not a verdict. The app must use the same feature names and types as this notebook.

In [15]:
```
example_transaction = pd.DataFrame([{feature: 0.0 for feature in FEATURES}])
example_transaction.loc[0, "Time"] = 100000.0
example_transaction.loc[0, "Amount"] = 850.0
example_probability = loaded.predict_proba(example_transaction)[0, list(loac
print(f"Estimated fraud probability: {example_probability:.1%}")
print(f"Review signal at selected threshold: {example_probability >= metadat
```

```
Estimated fraud probability: 0.0%
Review signal at selected threshold: False
```

✅ Final summary and reproducibility checklist

What we learned

- A Decision Tree partitions feature space through impurity-reducing questions.
- Gini impurity measures class mixing; depth and leaf-size controls limit overfitting.
- Accuracy alone is unsafe for rare fraud; precision, recall, F1 and PR-AUC reveal different trade-offs.
- A threshold is an operational policy that should be tied to documented costs.
- Leakage prevention and time-aware validation matter as much as the algorithm.
- A saved preprocessing-and-model pipeline is safer than saving a classifier without its transformations.

Before sharing this project

- [x] Synthetic dataset is deterministic and clearly labelled as synthetic.
- [x] Public dataset links and access/licensing cautions are documented.
- [x] Target leakage fields are excluded.
- [x] Train/test split is stratified and transformations fit on training data only.
- [x] Baseline, confusion matrix, precision, recall, F1, ROC-AUC and PR-AUC are reported.
- [x] Threshold trade-off and illustrative cost assumptions are visible.
- [x] Pipeline and metadata are saved and reloaded.
- [ ] Validate on an approved real dataset with time-based splitting.
- [ ] Complete privacy, fairness, governance and security review.
- [x] Streamlit profile and app directory: https://share.streamlit.io/user/abhishekpandaofficial.

Project tools

Python 3.12 · Jupyter · pandas · NumPy · Matplotlib · Seaborn · scikit-learn · joblib · nbconvert · Streamlit · Git/GitHub.

🚀 Streamlit



Browse the deployed project apps from the Streamlit profile and app directory.